

Performance Evaluation and Monte Carlo Stress-Testing of Relaxation FSAL Runge–Kutta Schemes

Application to Conservative Systems: Nonlinear Pendulum Analysis

Prachurja Dhar*

2105150@ugrad.cse.buet.ac.bd

Student ID: 2105150

Bangladesh University of Engineering
and Technology
Dhaka, Bangladesh

MD. Abir Hossain*

2105133@ugrad.cse.buet.ac.bd

Student ID: 2105133

Bangladesh University of Engineering
and Technology
Dhaka, Bangladesh

Asikur Rahman*

2105144@ugrad.cse.buet.ac.bd

Student ID: 2105144

Bangladesh University of Engineering
and Technology
Dhaka, Bangladesh

MD. Shadman Shafie*

2105134@ugrad.cse.buet.ac.bd

Student ID: 2105134

Bangladesh University of Engineering
and Technology
Dhaka, Bangladesh

Tousif Fahmeed Quadir*

2105127@ugrad.cse.buet.ac.bd

Student ID: 2105127

Bangladesh University of Engineering
and Technology
Dhaka, Bangladesh

Abstract

Explicit Runge–Kutta (RK) methods do not preserve invariants such as the energy of a conservative system, so the numerical energy drifts over long simulations. Relaxation removes this drift by rescaling every step with a parameter γ obtained from a scalar nonlinear equation, but combined with the first-same-as-last (FSAL) property it costs one extra right-hand-side (RHS) evaluation per step, which Blecke and Ranocha’s FSAL-R and R-FSAL schemes remove. We re-implement the baseline, naive, FSAL-R and R-FSAL variants in Python for the Bogacki–Shampine 3(2) and Dormand–Prince 5(4) pairs with PID step size control, verify them through reference-step, convergence-order and exact RHS-count checks, and apply them to the nonlinear pendulum. We extend the base study in two directions: a comparison of three root-finders for γ (Newton–Raphson, bisection and Algorithm 748) and a Monte Carlo stress test of 140,000 runs over 1000 random initial conditions in both motion regimes and seven tolerances. The relaxed methods kept the maximum energy drift below 1.2×10^{-12} in every run, whereas the baseline drifted by up to 0.19. At tolerance 10^{-6} , FSAL-R and R-FSAL matched the accuracy of naive relaxation (median error 2.0×10^{-6} against 4.6×10^{-5} for the third-order baseline) at baseline cost, while naive relaxation needed 4/3 and 7/6 times as many RHS evaluations. Newton’s method needed about 3.5 iterations per step, against 4.5 for Algorithm 748 and 42 for bisection, and failed far less often; relaxation failures concentrate in the rotation regime, where the fixed bracket $[0.8, 1.2]$ often contains no sign change.

CCS Concepts

• **Mathematics of computing** → **Ordinary differential equations**; *Nonlinear equations*; • **Computing methodologies** → Modeling and simulation.

Keywords

Relaxation Runge–Kutta methods, first-same-as-last (FSAL), invariant conservation, error-based step size control, root-finding, Monte Carlo simulation, nonlinear pendulum

1 Introduction and Base Paper Review

1.1 Background

We consider the initial value problem

$$u'(t) = f(u(t)), \quad u(t^0) = u^0, \quad (1)$$

equipped with an invariant η , a scalar function of the state whose value does not change along exact solutions:

$$\frac{d}{dt} \eta(u(t)) = \eta'(u) f(u) = 0. \quad (2)$$

Following the base paper, we call η the *entropy*; for the conservative system studied in this report it is the total energy. A frictionless pendulum, for example, keeps its energy $\eta(\omega, \theta) = \frac{1}{2}\omega^2 - \frac{g}{L} \cos \theta$ constant forever.

Explicit Runge–Kutta methods. An explicit s -stage Runge–Kutta (RK) method advances the solution from u^n to u^{n+1} over a step Δt by evaluating the right-hand side (RHS) f at intermediate stages and combining these slopes with weights b_i [8]:

$$y^i = u^n + \Delta t \sum_{j=1}^{i-1} a_{ij} f(y^j), \quad u^{n+1} = u^n + \Delta t \sum_{i=1}^s b_i f(y^i). \quad (3)$$

Each stage costs one evaluation of f . Since f is usually the most expensive part of a simulation, the cost of an ODE solver is measured in RHS evaluations.

Error-based step size control. A fixed Δt wastes work where the solution changes slowly and loses accuracy where it changes quickly. Embedded RK pairs such as the Bogacki–Shampine 3(2) pair (BS3) [6] and the Dormand–Prince 5(4) pair (DP5) [7] therefore reuse the same stages with a second set of weights $\hat{b}_i$ to form an embedded solution $\hat{u}^{n+1}$ of one order lower. The difference $u^{n+1} - \hat{u}^{n+1}$

*Group 4, Section C1. CSE 402: Numerical Analysis, Simulation and Modeling Sessional, Department of Computer Science and Engineering, BUET.

estimates the local error at no extra RHS cost, and a controller, here a PID controller [18, 19], uses the estimate to accept or reject the step and to choose the next Δt .

First-same-as-last (FSAL). BS3 and DP5 are FSAL methods: their last stage is evaluated at the new solution, $f(y^s) = f(u^{n+1})$, which is exactly the first stage $f(u^{n+1})$ of the next step. Reusing it saves one RHS evaluation per step, reducing the cost from 4 to 3 evaluations for BS3 and from 7 to 6 for DP5. The last weight of the main solution is $b_s = 0$; the last stage is still computed because the embedded solution uses it ($\widehat{b}_s \neq 0$).

Energy drift. Standard explicit RK methods do not preserve η . Every step makes a small truncation error, part of which moves η away from its correct value, and over a long integration these errors accumulate into a visible drift of the energy. For long-time simulations of conservative systems this is both a qualitative and a quantitative defect.

Relaxation. Relaxation Runge–Kutta methods [11, 15, 16] remove the drift with a small modification of the baseline method. After the ordinary step has produced u^{n+1} , the method searches along the line through u^n and u^{n+1} for the point with the correct entropy,

$$u_Y^{n+1} = u^n + \gamma (u^{n+1} - u^n), \quad \eta(u_Y^{n+1}) = \eta(u^n), \quad (4)$$

and continues the integration from u_Y^{n+1} at the relaxed time $t_Y^{n+1} = t^n + \gamma \Delta t$. The relaxation parameter γ is the root of a scalar nonlinear equation. Under mild assumptions it satisfies $\gamma = 1 + O(\Delta t^p)$, where p is the order of the baseline method, and the relaxed method keeps order p while conserving η exactly up to rounding [15]. The trivial root $\gamma = 0$, which would not advance the solution, is excluded. For Hamiltonian problems relaxation also slows the growth of the solution error over time [14].

The conflict. FSAL and relaxation do not combine directly. FSAL provides $f(u^{n+1})$ for the next step, but after relaxation the next step starts from u_Y^{n+1} , so it needs $f(u_Y^{n+1})$ instead. The straightforward (naive) remedy evaluates $f(u_Y^{n+1})$ afresh, which adds one RHS evaluation per step and cancels the FSAL saving exactly: BS3 returns to 4 evaluations per step and DP5 to 7. This efficiency penalty has been reported in performance studies of relaxation methods [10, 17].

1.2 Base Paper Review

Our project builds on the paper by Bleecke and Ranocha, *Relaxation Runge–Kutta Methods with First-Same-as-Last Structure* [5], published as a technical note in the *Journal of Scientific Computing* (vol. 106, article 18, 2026). A longer technical report [4] and a reproducibility repository written in Julia [3] accompany it. The paper asks how relaxation can be combined with FSAL and error-based step size control without the extra RHS evaluation of the naive approach.

Naive combination. The paper first formalises the approach used by software libraries such as OrdinaryDiffEq.jl [13]. The stages and the main solution are computed from the relaxed state u_Y^n ; the embedded solution uses the FSAL stage through its extra weight $\widehat{b}_{s+1}f(u^{n+1})$; the PID controller accepts or rejects the step; and only an accepted step is relaxed. The next step then needs $f(u_Y^{n+1})$, which costs one new RHS evaluation.

Table 1: The four variants studied in the base paper and re-implemented in this project. Costs are RHS evaluations per accepted step after the first.

Variant	Next step's $f(u_Y^{n+1})$	BS3	DP5
Baseline	no relaxation, $f(u^{n+1})$ reused	3	6
Naive	new evaluation	4	7
FSAL-R	interpolated by (5)	3	6
R-FSAL	exact; $f(u^{n+1})$ from (6)	3	6

FSAL-R. The first new method keeps this structure but replaces the new evaluation by a linear interpolation between two slopes that are already known:

$$f(u_Y^{n+1}) \approx f(u_Y^n) + \gamma (f(u^{n+1}) - f(u_Y^n)). \quad (5)$$

The authors prove that the interpolation error is $O(\Delta t^{p+1})$ and that the method therefore keeps the local order of both the main and the embedded solution. FSAL-R saves the extra evaluation and fits the structure of existing libraries, since relaxation is still performed only after a step has been accepted. Its weakness is that the next step starts from an approximate first stage.

R-FSAL. The second method changes the order of operations: it relaxes *before* the error estimate. Because $b_s = 0$, the main solution u^{n+1} does not depend on the last stage, so the method skips that stage, computes u_Y^{n+1} immediately, and spends the evaluation that the last stage would have cost at the relaxed point instead. The next first stage $f(u_Y^{n+1})$ is therefore exact. The skipped slope $f(u^{n+1})$, still needed by the embedded solution, is recovered by solving (5) for it:

$$f(u^{n+1}) \approx f(u_Y^n) + \frac{1}{\gamma} (f(u_Y^{n+1}) - f(u_Y^n)). \quad (6)$$

The error is then estimated with a relaxed embedded solution at t_Y^{n+1} , and the authors prove that it keeps the order of the embedded method. Table 1 summarises the four variants.

Experiments and findings. The published note demonstrates the methods on the Benjamin–Bona–Mahony (BBM) equation [2], a nonlinear dispersive wave equation, with a traveling-wave solution, periodic boundary conditions and a Fourier collocation discretisation that conserves a quadratic invariant. BS3 and DP5 are each run in the four variants of Table 1. The relaxation parameter is computed with the bracketing method Algorithm 748 of Alefeld, Potra and Shi [1] as implemented in Roots.jl [20]¹. In the work–precision diagrams, the naive method is more accurate than the baseline for a given tolerance but uses more RHS evaluations, while FSAL-R and R-FSAL reach the accuracy of the naive method without visibly increasing the work. The authors summarise this as an accuracy improvement of roughly 25% over the baseline at essentially the same cost. They found no clear winner between the two new methods for entropy-conservative problems; they recommend FSAL-R for general-purpose software because it does not change the core structure of a library, and R-FSAL for dissipative problems, where

¹The reproducibility code calls `Roots.AlefeldPotraShi()` with the bracket `[0.8, 1.2]`, a closely related bracketing method by the same authors.

the approximated first stage of FSAL-R is problematic. The technical report and the repository consider further problems, including the nonlinear pendulum integrated from the single initial state $(\omega, \theta) = (1.5, 1.0)$ on $t \in [0, 10]$.

Open questions. Three aspects of the base study motivate our work. First, each experiment uses a single initial condition, so it does not show how robust the methods are across the range of motions a system can perform. Second, γ is always computed with one bracketing method in a fixed bracket; how the choice of root-finder affects cost, attainable precision and robustness is not examined. Third, reading the reproducibility code showed that the branch intended to reject a step and halve Δt when no root lies in the bracket can never execute, because a later assignment always resets the relaxation flag. Such a step is instead accepted with $\gamma = 1$, that is, without relaxation, so energy conservation is silently lost on that step.

1.3 Objectives and Contributions

The objective of this project is to reproduce the base paper’s methods independently and to stress-test them beyond the single-trajectory experiments of the original study. Our contributions are as follows.

- **Independent re-implementation.** We implement the baseline, naive, FSAL-R and R-FSAL variants for BS3 and DP5 with the same PID controller in Python, using NumPy [9] and SciPy [21]. We verify them with reference-step, convergence-order and exact RHS-count checks.
- **Application to the nonlinear pendulum.** We apply all four variants to the nonlinear pendulum $\theta'' + \frac{g}{L} \sin \theta = 0$ and report its two qualitatively different regimes, libration (swinging) and rotation (passing over the top), separately.
- **Root-finder comparison.** We compare Newton–Raphson, bisection and Algorithm 748 for computing γ in terms of iterations per step, attainable energy conservation and failure rate.
- **Monte Carlo stress test.** We run 140,000 simulations over 1000 random initial conditions, seven tolerances from 10^{-3} to 10^{-9} , both tableaus and every method–root-finder combination, following the Monte Carlo approach of estimating behaviour from many random samples [12].
- **Robust failure handling.** When no root is found, our implementation rejects the step and halves Δt instead of silently accepting $\gamma = 1$, so that every accepted step conserves energy.

The remainder of this report is organised as follows. Section 2 describes the numerical methods and algorithms, Section 3 the implementation and experimental setup, Section 4 the results and their comparison with the base paper, and Section 5 our conclusions, limitations and directions for future work.

2 Methodology

This section specifies the numerical problem and every algorithm we implemented: the test problem and its invariant (Section 2.1), the embedded pairs and the step size controller shared by all methods (Section 2.2), the computation of the relaxation parameter with

Table 2: Embedded Runge–Kutta pairs used in this study.

Pair	Stages s	Order p	Embedded	$\widehat{b}_s$	PID $(\beta_1, \beta_2, \beta_3)$
BS3	4	3	2	$\frac{1}{8}$	$(0.6, -0.2, 0)$
DP5	7	5	4	$\frac{1}{40}$	$(0.7, -0.4, 0)$

three root-finders (Section 2.3), the four time-stepping loops (Section 2.4), and the quantities used to evaluate them (Section 2.5). The relaxation step (4) and the FSAL-R and R-FSAL approximations (5)–(6) are used exactly as introduced in Section 1.

2.1 Test Problem: The Nonlinear Pendulum

We study the frictionless nonlinear pendulum $\theta'' + \frac{g}{L} \sin \theta = 0$, where θ is the angle from the downward vertical. Introducing the angular velocity $\omega = \theta'$ gives a first-order system of the form (1) with state $u = (\omega, \theta)$:

$$u' = f(u) = \begin{pmatrix} -\frac{g}{L} \sin \theta \\ \omega \end{pmatrix}. \quad (7)$$

The component order (ω, θ) follows the reproducibility code [3], so the reference initial state $u^0 = (1.5, 1.0)$ means $\omega_0 = 1.5$ and $\theta_0 = 1.0$. Throughout we set $g/L = 1$. The invariant is the total energy, with its gradient

$$\eta(u) = \frac{1}{2} \omega^2 - \frac{g}{L} \cos \theta, \quad \nabla \eta(u) = (\omega, \frac{g}{L} \sin \theta). \quad (8)$$

Since $\nabla \eta(u) \cdot f(u) = -\omega \sin \theta + \omega \sin \theta = 0$, condition (2) holds. The gradient is needed only by Newton’s method (Section 2.3).

Motion regimes. With $g/L = 1$ the energy of the separatrix, the state balanced motionless at the top ($\theta = \pi, \omega = 0$), is $\eta = 1$. Initial states with $\eta(u^0) < 1$ swing back and forth (*libration*); states with $\eta(u^0) > 1$ pass over the top and rotate indefinitely (*rotation*). Since the two regimes have different solution behaviour, every initial condition is classified by its energy and all statistics are reported per regime rather than pooled. The reference state $(1.5, 1.0)$ has $\eta = 0.5847$ and is a libration.

Reference solution. To measure the true error, each initial condition is integrated once with the eighth-order Dormand–Prince method DOP853 [8] of SciPy [21] at relative tolerance 10^{-13} and absolute tolerance 10^{-14} , and its dense output is evaluated at the final time of each run. This plays the role of the Vern9 reference used by the authors [3] and is accurate to about 10^{-12} , three orders of magnitude below the error of the loosest tolerance we test.

2.2 Embedded Pairs and Step Size Control

Tableaus. We use the Bogacki–Shampine 3(2) pair (BS3) [6] and the Dormand–Prince 5(4) pair (DP5) [7], with the coefficients of the reproducibility code. Their properties are listed in Table 2. Both are FSAL: the last row of A equals b and $c_s = 1$, so the last stage is evaluated at u^{n+1} , and $b_s = 0$ while $\widehat{b}_s \neq 0$. Every coefficient is stored once as an exact rational number, from which the floating-point tableau used by the solvers is derived. For BS3 the main weights are $b = (\frac{2}{9}, \frac{1}{3}, \frac{4}{9}, 0)$ and the embedded weights $\widehat{b} = (\frac{7}{24}, \frac{1}{4}, \frac{1}{3}, \frac{1}{8})$.

Error estimate. The local error of a step from u^n is estimated by the scaled root-mean-square difference between the main and embedded solutions,

$$\text{err} = \left(\frac{1}{N} \sum_{i=1}^N \left(\frac{u_i^{n+1} - \widehat{u}_i^{n+1}}{\tau_a + \tau_r \max(|u_i^{n+1}|, |u_i^n|)} \right)^2 \right)^{1/2}, \quad (9)$$

where $N = 2$ is the dimension of the state and τ_a and τ_r are the absolute and relative tolerances. As in the base study, we always set $\tau_a = \tau_r = \text{tol}$. A value $\text{err} \leq 1$ means that the step meets the requested tolerance.

PID controller. Following the reproducibility code, the controller stores the reciprocals $\epsilon_j = 1/\text{err}_j$ of the current and the two previous error estimates and computes the step size factor

$$x = \epsilon_1^{\beta_1/k} \epsilon_2^{\beta_2/k} \epsilon_3^{\beta_3/k}, \quad \varphi = 1 + \arctan(x - 1), \quad (10)$$

with $k = p$ the order of the method and the gains of Table 2 [18, 19]. The limiter in (10) leaves factors near 1 almost unchanged and caps growth at $1 + \pi/2 \approx 2.57$. A step is accepted if $\varphi \geq 0.81$, and the next step size is $\Delta t \leftarrow \varphi \Delta t$. The error history starts at $\epsilon_2 = \epsilon_3 = 1$ and is shifted only when a step is finally accepted. This separation matters for the relaxation methods, because a step that passes the error test can still be rejected when no relaxation parameter is found.

Initial step size. The first step size is chosen by the standard heuristic of Hairer, Nørsett and Wanner [8], as in the reproducibility code. With the component scale $\text{sk}_i = \tau_a + \tau_r |u_i^0|$ and $\|\cdot\|$ the Euclidean norm of the scaled vector, it computes

$$d_0 = \|u^0/\text{sk}\|, \quad d_1 = \|f(u^0)/\text{sk}\|, \quad \Delta t_0 = 10^{-2} d_0/d_1,$$

$$d_2 = \|(f(u^0 + \Delta t_0 f(u^0)) - f(u^0))/\text{sk}\|/\Delta t_0,$$

$$\Delta t_1 = 10^{-(2+\log_{10} \max(d_1, d_2))/p},$$

$$\Delta t = \min(100 \Delta t_0, \Delta t_1, t_{\text{end}} - t^0). \quad (11)$$

It costs two RHS evaluations. The first, $f(u^0)$, is reused as the first stage of the first step.

2.3 Computing the Relaxation Parameter

For a step from u^n to u^{n+1} with direction $d = u^{n+1} - u^n$, the relaxation condition in (4) is the scalar root problem

$$r(\gamma) = \eta(u^n + \gamma d) - \eta(u^n) = 0, \quad r'(\gamma) = \nabla \eta(u^n + \gamma d) \cdot d. \quad (12)$$

Evaluating r requires only η , never f , so computing γ adds no RHS evaluations regardless of the root-finder. Since $r(0) = 0$ for every step, the search is restricted to the bracket $[0.8, 1.2]$ of the reproducibility code, which contains the root $\gamma = 1 + \mathcal{O}(\Delta t^{p-1})$ of interest [15] and excludes the trivial one. All root-finders share the interface $(r, r', [\gamma_{\min}, \gamma_{\max}], x_{\text{tol}})$ with $x_{\text{tol}} = 10^{-14}$ and return the root, the iteration count and a convergence flag.

Newton–Raphson. Starting from $\gamma_0 = 1$, the iteration $\gamma_{m+1} = \gamma_m - r(\gamma_m)/r'(\gamma_m)$ converges quadratically from this very good initial guess. It needs the gradient in (8) and has no convergence guarantee. The usual stopping rule $|\gamma_{m+1} - \gamma_m| \leq x_{\text{tol}}$ is not always reachable: r is the difference of two nearly equal energies and therefore carries a rounding error of a few ϵ_{mach} , so the attainable accuracy in γ is about $\epsilon_{\text{mach}}/|r'|$, and r' shrinks with the step size. For BS3 at tolerance 10^{-7} , for example, $|r'| \approx 10^{-6}$, so the Newton

Input: tableau, tol, root-finder. **Start:** $\Delta t, k_1 = f(u^0)$ from (11).

while $t < t_{\text{end}}$:

- (1) Compute stages $k_2, \dots, k_s$ ($s - 1$ RHS evaluations).
- (2) Form u^{n+1} with b and $\widehat{u}^{n+1}$ with $\widehat{b}$; compute err (9) and φ (10).
- (3) If $\varphi < 0.81$: reject the step and go to step 6.
- (4) *Baseline*: accept; u^{n+1} at $t + \Delta t$; next $k_1 \leftarrow k_s$.
- (5) *Naive, FSAL-R*: solve (12) for γ .
 - On failure: reject, $\Delta t \leftarrow \Delta t/2$, and repeat the step.
 - Otherwise accept; u^{n+1} at $t + \gamma \Delta t$. Next k_1 : *naive* $f(u^{n+1})$ (one extra RHS evaluation); *FSAL-R* $k_1 + \gamma(k_s - k_1)$, as in (5).
- (6) $\Delta t \leftarrow \varphi \Delta t$.

Figure 1: Common loop of the baseline, naive and FSAL-R methods.

correction stalls near 2×10^{-10} , far above x_{tol} . We therefore also stop when the correction no longer decreases, which marks this rounding floor. A result outside the bracket, or a vanishing or non-finite derivative, is reported as not converged. The iteration limit is 50.

Bisection. Bisection requires $r(\gamma_{\min})$ and $r(\gamma_{\max})$ to have opposite signs and halves the bracket until its width is below x_{tol} , which takes about 45 iterations for the bracket width 0.4. It needs no derivative and is guaranteed to converge when the sign condition holds; otherwise it reports failure.

Algorithm 748. Algorithm 748 of Alefeld, Potra and Shi [1] combines inverse cubic interpolation and Newton-like steps with bracket reduction, so it keeps the guarantee of bisection while converging almost as fast as Newton. We use its SciPy implementation, `toms748`. As the algorithm used by the base study, it serves as the reference for the other two. Like bisection, it requires a sign change at the bracket ends.

Acceptance of a root. All solvers obtain γ through one shared function, so that the bracket and the tolerances are identical across methods. The returned value is accepted only if the root-finder reports convergence and the residual, recomputed independently, satisfies $|r(\gamma)| \leq 10^{-12}$. If the root-finder fails but $|r(1)| \leq 10^{-12}$, the step already conserves the energy to rounding accuracy, and $\gamma = 1$ is returned as a verified root. This mirrors the special case in the reproducibility code that takes $\gamma = 1$ when the residual vanishes at both bracket ends. In every other case the computation is reported as a *relaxation failure*, and the solver rejects the step and halves Δt (Section 2.4). A step is never continued with an unverified $\gamma = 1$.

2.4 Time-Stepping Algorithms

The four variants of Table 1 share the stage computation, the error estimate (9) and the controller (10). Baseline, naive and FSAL-R follow the same loop and differ only in how the next first stage is obtained after an accepted step (Figure 1). R-FSAL reorders the step (Figure 2). In both loops, k_1 denotes the cached first stage and t_{end} the final time; a step that would pass t_{end} is shortened to end there.

Relaxed time and the final step. An accepted relaxed step advances time to $t + \gamma \Delta t$. The only exception is the last step: when

```

while  $t < t_{\text{end}}$ :
  (1) Compute stages  $k_2, \dots, k_{s-1}$  only ( $s - 2$  RHS evaluations);
      form  $u^{n+1}$ , which does not need  $k_s$  because  $b_s = 0$ .
  (2) Solve (12) for  $\gamma$ . On failure: reject,  $\Delta t \leftarrow \Delta t/2$ , and repeat the
      step.
  (3) Form  $u_Y^{n+1}$  at  $t_Y^{n+1} = t + \gamma \Delta t$  and evaluate  $f(u_Y^{n+1})$  (one RHS
      evaluation).
  (4) Reconstruct  $k_s \approx k_1 + \gamma^{-1}(f(u_Y^{n+1}) - k_1)$  as in (6).
  (5) Form the relaxed embedded solution  $\hat{u}_Y^{n+1} = u^n + \gamma \Delta t \sum_i \hat{b}_i k_i$ 
      and compute err from  $u_Y^{n+1}$  and  $\hat{u}_Y^{n+1}$ .
  (6) If  $\varphi \geq 0.81$ : accept; next  $k_1 \leftarrow f(u_Y^{n+1})$ , which is exact. Other-
      wise reject.
  (7)  $\Delta t \leftarrow \varphi \Delta t$ .

```

Figure 2: The R-FSAL loop: relaxation precedes the error estimate.

$t + \Delta t$ equals t_{end} up to a relative tolerance of $\sqrt{\varepsilon_{\text{mach}}} \approx 1.49 \times 10^{-8}$, the state is relaxed but the time is set to t_{end} , so that every run ends exactly where the reference solution is evaluated. The same tolerance is used by all four methods.

Relaxation failures. When no valid γ is found, the step is rejected and Δt is halved before retrying from the same u^n . The controller history is not shifted. The reproducibility code intends the same behaviour, but in that code the branch is unreachable (Section 1.2); our implementation executes it. The run aborts with an error if Δt falls below 10^{-14} , so a failing configuration is reported rather than silently truncated.

RHS evaluation count. The first stage of every attempt comes from the cache, and a rejected step restarts from the same u^n with a still-valid cache. Hence every attempt, accepted or rejected, costs $s - 1$ evaluations. With c_0 evaluations spent on the initial step size ($c_0 = 2$ for the heuristic (11), 1 for a prescribed Δt), n_{acc} accepted and n_{rej} rejected attempts, and n_{fail} relaxation failures (included in n_{rej}), the total number of RHS evaluations is

$$\begin{aligned}
n_f^{\text{baseline}} &= n_f^{\text{FSAL-R}} = c_0 + (s - 1)(n_{\text{acc}} + n_{\text{rej}}), \\
n_f^{\text{naive}} &= c_0 + (s - 1)(n_{\text{acc}} + n_{\text{rej}}) + n_{\text{acc}}, \\
n_f^{\text{R-FSAL}} &= c_0 + (s - 1)(n_{\text{acc}} + n_{\text{rej}}) - n_{\text{fail}}.
\end{aligned} \tag{13}$$

The last term appears because an R-FSAL attempt that fails to relax stops before its evaluation at u_Y^{n+1} and costs only $s - 2$ evaluations. These identities hold exactly for every run and serve as a correctness check of the FSAL bookkeeping (Section 3.3).

2.5 Evaluation Metrics

Each run, defined by an initial condition, a method, a tableau, a tolerance and a root-finder, is integrated over $t \in [0, 10]$ and summarised by the following quantities.

- **Cost:** the number of RHS evaluations n_f , together with n_{acc} , n_{rej} and n_{fail} .
- **Global error:** the Euclidean distance $\|u_{\text{final}} - u_{\text{ref}}(t_{\text{final}})\|_2$ between the final state and the reference solution.
- **Energy drift:** the maximum over all accepted steps of $|\eta(u^n) - \eta(u^0)|$, and its value at the final time.

Table 3: Modules of the implementation (package src).

Module	Responsibility
contracts.py	Shared types: state, problem, tableau, root and solver results
problems.py	Pendulum RHS, η , $\nabla \eta$, regime classification, DOP853 reference, initial-condition sampling
tableaus.py	BS3 and DP5 as exact fractions, order-condition checks, PID gains
stepping.py	Error estimate, PID controller, initial step size
rootfind.py	Newton, bisection, Algorithm 748, shared γ solver
solvers/classic.py	Baseline, naive and FSAL-R in one loop
solvers/rfsal.py	R-FSAL
montecarlo.py	Parallel sweep over all configurations; CSV output
analysis.py	Aggregation, work-precision diagrams, tables

- **Root-finding cost:** the total number of root-finder iterations, reported per accepted step.
- **Relaxation parameter:** the minimum and maximum of γ and the mean and maximum of $|\gamma - 1|$, which by the theory of [15] should decrease as the tolerance tightens.

Accuracy is compared against cost in work-precision diagrams, as in the base paper [5]. Over many initial conditions, each point is the median over the initial conditions of one regime, and the 10th to 90th percentile range shows the spread across initial conditions.

3 Implementation and Experiments

This section describes how the methods of Section 2 were implemented (Section 3.1), where the implementation deliberately departs from the reproducibility code (Section 3.2), how it was verified before any experiment was run (Section 3.3), and how the Monte Carlo experiments were set up (Section 3.4).

3.1 Software Design

All methods were implemented from scratch in Python 3, porting the solver loops of the authors' Julia code [3]. We did not use SciPy's general-purpose integrator `solve_ivp`, because it provides no way to replace the step size controller or to modify a step after it has been computed, both of which relaxation requires. SciPy [21] supplies only Algorithm 748 and the DOP853 reference solution; NumPy [9] is used for random sampling, and pandas and Matplotlib for analysis. Table 3 lists the modules.

The following design decisions shaped the implementation.

- **Scalar state.** The state is a pair of Python floats, and the RHS uses `math.sin`. For a two-component state, the per-call overhead of NumPy arrays outweighs the arithmetic, so plain floats are considerably faster in the inner loop.
- **Exact tableaus.** Each Butcher coefficient is written once as a Fraction; the float tableau is generated from it, and

the main weights b are taken from the last row of A rather than typed again. The tests check the order conditions in exact arithmetic, so a mistyped digit fails a test instead of silently lowering the order.

- **One loop for three methods.** Baseline, naive and FSAL-R share a single loop that branches only after an accepted step (Figure 1). Any difference between their results can therefore come only from the treatment of the first stage. R-FSAL, whose step is ordered differently, has its own module.
- **Shared root-finding entry point.** Every solver obtains γ from the same function with the same bracket and tolerances. A γ differing in its last digits can flip a borderline accept/reject decision, after which two runs follow different step sequences; a shared configuration rules this out as a source of difference between methods.
- **Exact cost accounting.** Each solver counts its RHS evaluations explicitly, starting from the count c_0 reported by the initial step size routine instead of a hard-coded constant. Tests compare the reported count with a wrapper that counts actual calls of f .
- **Explicit failure.** A step size below 10^{-14} or more than 10^7 attempts raises an exception instead of returning a truncated result.

3.2 Deviations from the Reproducibility Code

Transcribing the Julia code revealed several points where a literal port would have produced an unfair comparison or an incorrect method. Table 4 lists our departures; each is marked by a comment at the corresponding line of our code.

The change to the Newton stopping rule was found during development: with only the textbook rule, Newton reached a residual of 2.2×10^{-16} but kept iterating until its iteration limit and then reported failure, which produced 653 spurious relaxation failures in a single run at tight tolerance. Algorithm 748 and bisection, whose stopping criteria depend on the bracket width, did not need this change.

3.3 Verification

The project was built in consecutive steps, one module at a time. Before a module was handed on, it had to pass its own tests and a set of *verification gates*, pass/fail checks of correctness that later modules relied on. Every module has its own test file. The final test suite contains 175 passing tests; the comparison against reference outputs of the Julia code (described below) was not carried out.

Reference step. All modules were checked against one BS3 step computed independently by hand, from $u^0 = (1.5, 1.0)$ with $\Delta t = 0.4$. Table 5 lists its values; the code reproduces each of them to the printed precision. In particular, the second component of the FSAL-R interpolation equals the true $f(u_V^1)$ exactly, because that component of f , $\theta' = \omega$, is linear in u , whereas the first component, $-\sin \theta$, is not.

Gates. The gates check properties that the experiments depend on. G1–G4 use the reference initial state, $t \in [0, 10]$ and Algorithm 748.

- **G1, root-finders.** Newton, bisection and Algorithm 748 return the same $\gamma = 1.0036353183$ for the reference step, using 4, 45 and 4 iterations respectively. They also agree on test functions with known roots.
- **G2, convergence order.** With fixed steps and no relaxation, halving Δt must reduce the global error by 2^p . For BS3 with 32 to 256 steps the observed ratios are 7.58, 7.92 and 7.99 (expected 8); for DP5 with 64 to 512 steps they are 41.8, 35.6 and 33.5 (expected 32). The step ranges are chosen per method to stay above the rounding floor.
- **G3, RHS count.** The identities (13) must hold exactly in every run. For example, at tolerance 10^{-6} BS3 takes 203 accepted and no rejected steps, and the counts are $2+3 \cdot 203 = 611$ for baseline, FSAL-R and R-FSAL and $611 + 203 = 814$ for naive; DP5 takes 37 accepted and 2 rejected steps, giving 236 for the three FSAL methods and 273 for naive.
- **G4, energy conservation.** The relaxed methods must keep the energy at rounding level. At tolerance 10^{-6} the maximum energy drift is 8.1×10^{-6} (BS3) and 6.6×10^{-6} (DP5) for the baseline, and below 10^{-15} for R-FSAL with both tableaus.
- **G6, relaxation parameter.** In a Monte Carlo test, γ stays close to 1 and $|\gamma - 1|$ decreases as the tolerance tightens, as $\gamma = 1 + O(\Delta t^{p-1})$ predicts.

Gate G5 was to compare our RHS counts step by step with values produced by running the authors' Julia code. The Julia reference values were never generated, so G5 was not performed; our implementation is verified by the checks above rather than against the original program.

Further checks. Additional tests confirm the remaining parts of the methods:

- The FSAL-R interpolation is one order more accurate than reusing k_s unchanged. For one BS3 step, the error of the cached value decreases by about 16 per halving of Δt , against 8 without interpolation, as the paper's Lemma 1 predicts.
- The R-FSAL extrapolation reproduces $f(u^{n+1})$ to rounding error for a linear RHS (the harmonic oscillator), and R-FSAL never computes the last stage directly.
- When the root-finder is made to fail, the solver rejects the step, halves Δt and continues without accepting an unrelaxed step, and the RHS identity still holds.
- The pendulum period agrees with the elliptic-integral formula, and the reference solution conserves the energy.
- The Monte Carlo sweep writes identical results for the same seed, whether it runs serially or in parallel.

3.4 Experimental Setup

Initial conditions. We sample $N = 1000$ initial states uniformly from $\theta_0 \in [-3, 3]$ and $\omega_0 \in [-2, 2]$ with the NumPy generator `default_rng` and seed 0. This rectangle contains states on both sides of the separatrix: the sampled energies range from -0.9999 to 2.89. Each state is classified by its energy, which yields 648 libration and 352 rotation states. Fixing the seed makes the set of initial conditions, and with it the whole sweep, reproducible.

Table 4: Deliberate departures from the authors’ reproducibility code.

Item	Reproducibility code	Our implementation
Baseline bookkeeping	Accepted baseline steps are not counted, and the PID error history is never shifted, so the baseline runs on a weaker controller than the relaxation methods.	Accepted steps are counted and the history is shifted, so all four methods use the same controller.
Relaxation failure	The branch that should reject the step and halve Δt cannot execute, because the failure flag is reset beforehand; the step proceeds with $\gamma = 1$.	The step is rejected and Δt is halved; no unrelaxed step is ever accepted (Section 2.4).
R-FSAL failure cost	—	A failed R-FSAL attempt costs $s - 2$ evaluations; this is accounted for by the term $-n_{\text{fail}}$ in (13).
Newton stopping rule	Not applicable (Algorithm 748 only).	Newton also stops when its correction stops decreasing, avoiding spurious failures at the rounding floor (Section 2.3).
Already-conserved steps	$\gamma = 1$ if the residual vanishes at both bracket ends.	$\gamma = 1$ only if $ r(1) \leq 10^{-12}$, checked like any other root.
Final step	Detected by an approximate comparison with an implicit tolerance.	Explicit relative tolerance 1.49×10^{-8} , shared by all methods.
Reference solution	Vern9 at tolerance 10^{-14} (Julia).	DOP853 at relative tolerance 10^{-13} , absolute 10^{-14} (SciPy).

Table 5: Reference BS3 step from $u^0 = (1.5, 1.0)$ with $\Delta t = 0.4$.

Quantity	Value
u^1 (weights b)	(1.1215519, 1.5261710)
$\hat{u}^1$ (weights $\hat{b}$)	(1.1241401, 1.5257058)
err (9), tol 10^{-3}	0.7436 (accepted)
γ (all three root-finders)	1.0036353183
u_γ^1	(1.1201761, 1.5280838)
t_γ^1	0.4014541
$f(u_\gamma^1)$, exact	(−0.9990880, 1.1201761)
$f(u_\gamma^1)$ by FSAL-R (5)	(−0.9995771, 1.1201761)
$f(u^1)$ by R-FSAL (6)	(−0.9985170, 1.1215519)

Table 6: Factors of the Monte Carlo sweep.

Factor	Levels
Initial conditions	1000 (648 libration, 352 rotation), seed 0
Tableau	BS3, DP5
Tolerance	$10^{-3}, 10^{-4}, \dots, 10^{-9}$ ($\tau_a = \tau_r$)
Method	baseline, naive, FSAL-R, R-FSAL
Root-finder	Newton, bisection, Algorithm 748 (relaxed methods only)
Time interval	$[0, 10]$, $g/L = 1$

Configurations. Each initial condition is integrated over $t \in [0, 10]$ with $g/L = 1$ in every combination of the factors in Table 6. The baseline uses no root-finder, so it is run once per tableau and tolerance instead of once per root-finder, which avoids duplicate runs. This gives $2 \times 7 \times (1 + 3 \times 3) = 140$ runs per initial condition and 140,000 runs in total. The seven tolerances match the range of the base study’s work–precision diagrams.

Execution. Before the full sweep, a smoke test with 50 initial conditions was run and inspected. The full sweep distributes the initial conditions over worker processes with Python’s multiprocessing. Each worker builds the DOP853 reference once per initial condition and reuses it for all 140 configurations. Results are written in a fixed order, so the output does not depend on the number of workers. Any run that raises an exception is recorded as a failed row instead of stopping the sweep. The full sweep took 76.5 minutes on two CPU cores, and all 140,000 runs completed.

Recorded data. Each run produces one CSV row containing its configuration, the regime and energy of its initial state, the final time and state, and the metrics of Section 2.5: n_f , n_{acc} , n_{rej} , n_{fail} , root-finder iterations, global error, final and maximum energy drift, and summary statistics of γ . Full trajectories are not stored. To show the energy over time, separate runs from the reference state at tolerance 10^{-6} with Algorithm 748 record the full history.

Analysis. The analysis script groups the rows by regime, tableau, root-finder, method and tolerance. For each group it computes the median and the 10th and 90th percentiles of n_f and of the global error. It then draws work–precision diagrams with one point per tolerance and tabulates the root-finder iterations per accepted step, the energy drift and the relaxation failures. Failed runs, of which there were none, would be counted separately and excluded from these statistics. Section 4 presents the results.

4 Results and Discussion

The complete Monte Carlo sweep finished all 140,000 runs. The principal quantitative findings are collected in Table 7; the figures then separate invariant preservation from the accuracy–cost tradeoff. Unless otherwise stated, the work–precision statistics are medians over the initial conditions in each motion regime, with shaded bands spanning the 10th to 90th percentiles.

Table 7: Summary of the principal experimental results. The errors at tolerance 10^{-6} refer to BS3; root iterations are per accepted step.

Quantity	Result
Completed Monte Carlo runs	140,000
Largest baseline energy drift	0.19
Largest relaxed-method energy drift	$< 1.2 \times 10^{-12}$
Median baseline error, $\text{tol} = 10^{-6}$	4.6×10^{-5}
Median relaxed error, $\text{tol} = 10^{-6}$	2.0×10^{-6}
Naive/baseline RHS cost, BS3	4/3
Naive/baseline RHS cost, DP5	7/6
Newton iterations per step	3.5
Algorithm 748 iterations per step	4.5
Bisection iterations per step	42

4.1 Energy Conservation

Figure 3 shows the energy history for the reference initial condition at tolerance 10^{-6} . With both tableaux, the baseline accumulates visible energy error, reaching approximately 10^{-5} during this single trajectory. In contrast, naive relaxation, FSAL-R and R-FSAL remain near the floating-point rounding level, generally between 10^{-16} and 10^{-15} . The occasional downward spikes are not losses of accuracy: they occur when the computed energy happens to be even closer to its initial value.

The Monte Carlo results confirm that this behaviour is not specific to the reference trajectory. Across both libration and rotation, every accepted relaxed step satisfies the invariant condition, and the largest drift over all relaxed runs is below 1.2×10^{-12} . The baseline has no such constraint and reaches a drift of 0.19. Thus, the rejection-and-retry rule for a failed relaxation solve succeeds in protecting the invariant even in the more difficult rotation regime.

4.2 Accuracy versus Computational Work

Figure 4 reports final-solution error against the number of RHS evaluations. For BS3, all three relaxation variants lie consistently below the baseline: at tolerance 10^{-6} their median error is 2.0×10^{-6} , compared with 4.6×10^{-5} for the baseline. This advantage appears in both motion regimes, although rotation has a wider 10–90% spread because trajectories near and above the separatrix are more sensitive to phase error.

For DP5 the four curves are closer because the fifth-order baseline already has a small truncation error. Nevertheless, R-FSAL is usually the lowest curve at moderate and tight tolerances, while FSAL-R and naive relaxation remain competitive. The improvement is again obtained without changing the formal order: relaxation reduces the error constant while enforcing the energy constraint.

The cost difference between the relaxation variants follows directly from their FSAL bookkeeping. Naive relaxation evaluates the RHS once more after every accepted step, giving asymptotic cost ratios of 4/3 for BS3 and 7/6 for DP5 relative to the baseline. FSAL-R and R-FSAL remove that extra call; their points therefore reach essentially the accuracy of naive relaxation at baseline RHS cost. This reproduces the qualitative result of Bleecke and Ranocha [5],

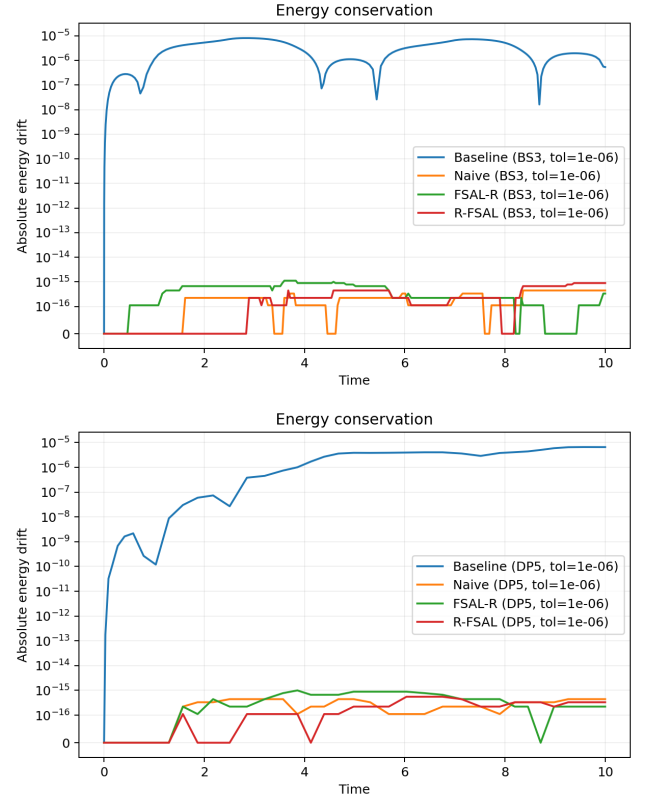


Figure 3: Absolute energy drift over time for the reference pendulum state, tolerance 10^{-6} and Algorithm 748: BS3 (top) and DP5 (bottom).

shown in Figure 5, and extends it from one trajectory to 1000 initial conditions in two motion regimes.

4.3 Effect of the Root-Finder

Changing the root-finder has little effect on the position of the work–precision curves. This is expected because evaluating the scalar energy residual does not call the ODE RHS, and all three algorithms target the same nontrivial root near $\gamma = 1$. It does, however, affect scalar work and robustness. Newton is fastest, requiring about 3.5 iterations per accepted step, followed by Algorithm 748 at 4.5; bisection needs about 42 iterations because it reduces the fixed bracket linearly.

Newton also fails less often because it begins at the accurate initial guess $\gamma = 1$ and does not require opposite signs at the end-points. Bisection and Algorithm 748 both require a sign change in $[0.8, 1.2]$. Their relaxation failures are concentrated in rotation cases, where that fixed interval more often misses the desired root. These failures do not corrupt the solution: the attempted step is rejected, its size is halved, and relaxation is tried again. They only add work. Consequently, Newton is the best choice for this pendulum experiment when the analytic gradient is available, while Algorithm 748 remains the safer derivative-free choice.

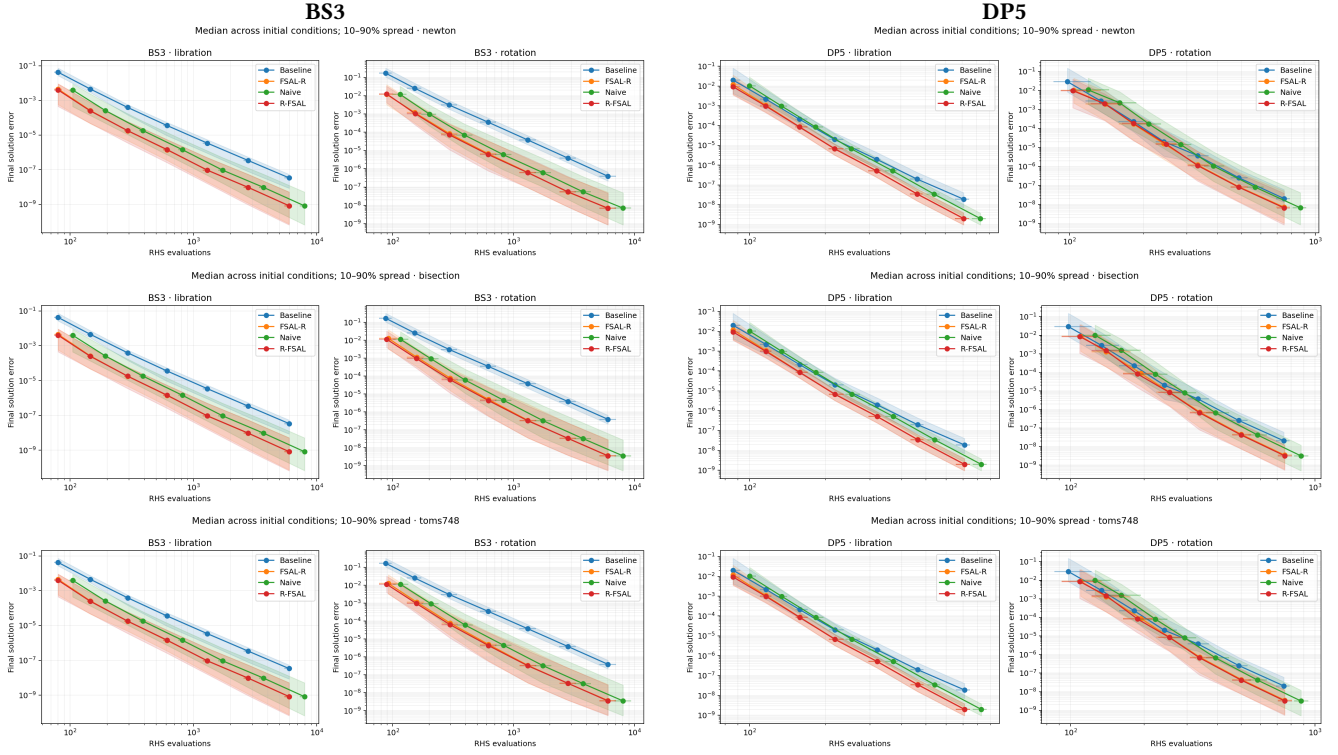


Figure 4: Work–precision results for BS3 (left column) and DP5 (right column), using Newton (first row), bisection (second row) and Algorithm 748 (third row). Each plot separates libration and rotation.

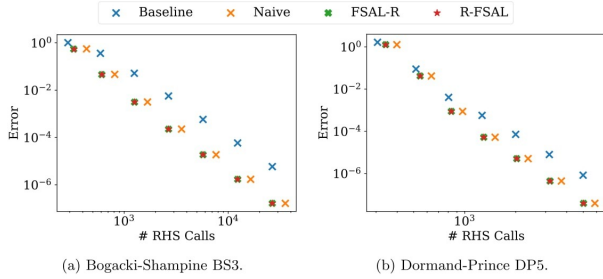


Figure 5: Work–precision comparison reported in the base study [5].

Overall, the experiments support the central claim of the FSAL relaxation schemes: FSAL-R and R-FSAL combine the invariant preservation and improved accuracy of naive relaxation with essentially the RHS cost of the baseline. The benefit is clearest for BS3, but it remains visible for DP5 and persists across both libration and rotation.

5 Conclusion and Future Work

This project independently implemented and evaluated the baseline, naive relaxation, FSAL-R and R-FSAL Runge–Kutta schemes for the nonlinear pendulum. The implementation was verified through

a reference step, convergence-order tests, exact RHS-evaluation counts and invariant-conservation checks. The subsequent Monte Carlo study extended the single-trajectory experiments of the base paper to 1000 initial conditions, covering both libration and rotation, seven tolerances, two embedded RK pairs and three root-finders.

The results show that relaxation provides a substantial improvement in long-time reliability. All relaxed variants preserved the pendulum energy to near machine precision, with maximum drift below 1.2×10^{-12} over the complete sweep, whereas the baseline reached a drift of 0.19. Relaxation also improved final-state accuracy, most clearly for BS3: at tolerance 10^{-6} , the median error decreased from 4.6×10^{-5} for the baseline to 2.0×10^{-6} for the relaxed schemes. Naive relaxation achieved this at the cost of an additional RHS evaluation per accepted step. FSAL-R and R-FSAL removed that overhead and attained essentially the same accuracy at baseline RHS cost. These observations reproduce the principal finding of Bleecke and Ranocha [5] and show that it continues to hold over a much broader set of pendulum trajectories.

The root-finder comparison further showed that preserving the invariant does not require an expensive nonlinear solve. Newton’s method required about 3.5 iterations per accepted step and was the most efficient option when the energy gradient was available. Algorithm 748 required about 4.5 iterations and offered a robust derivative-free alternative, while bisection required about 42 iterations. Bracketing failures occurred mainly for rotating solutions because the fixed interval $[0.8, 1.2]$ did not always contain a sign

change. Rejecting such attempts and reducing the step size ensured that no unrelaxed step was silently accepted.

The study has several limitations. It considers one conservative system in two dimensions and measures work primarily by RHS evaluations; consequently, the conclusions do not yet include large systems, expensive invariant evaluations or parallel hardware. The fixed relaxation bracket is inherited from the reproducibility study rather than adapted to the local step, and the reference solution, although computed at tight tolerance, is still numerical. Furthermore, FSAL-R and R-FSAL were tested only with BS3 and DP5 over the finite interval $[0, 10]$.

Future work should therefore examine the following directions.

- Develop an adaptive strategy for locating and bracketing the nontrivial relaxation root, especially for rotation and near-separatrix trajectories. This could reduce rejected attempts without weakening the invariant check.
- Test the methods on higher-dimensional Hamiltonian and entropy-conservative problems, including semidiscretized partial differential equations, where the cost of an RHS evaluation makes the FSAL saving more significant.
- Measure wall-clock time, memory traffic and parallel scalability in addition to RHS counts, so that the scalar root-finding and invariant evaluation costs are represented directly.
- Study substantially longer integration intervals and initial states closer to the separatrix to quantify phase error, accumulated rejections and the long-time behaviour of the cached FSAL stages.
- Compare further embedded RK pairs and root solvers, and investigate automatic selection between Newton and a safeguarded derivative-free method when gradient information is unavailable or unreliable.
- Extend the failure-handling analysis to dissipative and entropy-stable systems, for which the relative advantages of FSAL-R and R-FSAL may differ from those observed for the conservative pendulum.

In summary, relaxation is effective not only as an invariant-correction mechanism but also as a practical accuracy improvement. Among the tested variants, FSAL-R and R-FSAL provide the strongest overall tradeoff: they retain near-machine-precision energy conservation and the accuracy of naive relaxation without increasing the leading RHS cost of the underlying FSAL Runge–Kutta method.

6 Contribution of each member

Table 8 summarizes the primary responsibilities of each group member. All members also participated in reviewing the final implementation and report.

Table 8: Contribution of each group member.

Member (student ID)	Primary responsibilities
Prachurja Dhar (lead) 2105150	Shared interfaces (<code>contracts.py</code>), pendulum problem (<code>problems.py</code>), algorithm specification (<code>docs/ALGORITHMS.md</code>) and report preparation.
MD. Abir Hossain 2105133	Butcher tableaus (<code>tableaus.py</code>), error estimation and PID controller (<code>stepping.py</code>).
MD. Shadman Shafie 2105134	Root-finders (<code>rootfind.py</code>) and baseline, naive and FSAL-R solvers (<code>solvers/classic.py</code>).
Asikur Rahman 2105144	R-FSAL solver (<code>solvers/rfsal.py</code>) and Monte Carlo sweep (<code>montecarlo.py</code>).
Tousif Fahmeed Quadir 2105127	Analysis, figures and summary tables (<code>analysis.py</code>).

github link : https://github.com/Legenddipro/Numerical_Project

References

- [1] Götz E. Alefeld, Florian A. Potra, and Yixun Shi. 1995. Algorithm 748: Enclosing Zeros of Continuous Functions. *ACM Trans. Math. Software* 21, 3 (1995), 327–344. doi:10.1145/210089.210111
- [2] Thomas Brooke Benjamin, Jerry L. Bona, and John J. Mahony. 1972. Model Equations for Long Waves in Nonlinear Dispersive Systems. *Philosophical Transactions of the Royal Society of London. Series A, Mathematical and Physical Sciences* 272, 1220 (1972), 47–78. doi:10.1098/rsta.1972.0032
- [3] Sebastian Bleecke and Hendrik Ranocha. 2023. Reproducibility Repository for “Relaxation Runge–Kutta Methods with First-Same-as-Last Structure”. https://github.com/ranocha/2023_FSAL_relaxation. doi:10.5281/zenodo.10201246
- [4] Sebastian Bleecke and Hendrik Ranocha. 2023. Step Size Control for Explicit Relaxation Runge–Kutta Methods Preserving Invariants. arXiv:2311.14050 [math.NA] doi:10.48550/arXiv.2311.14050
- [5] Sebastian Bleecke and Hendrik Ranocha. 2026. Relaxation Runge–Kutta Methods with First-Same-as-Last Structure. *Journal of Scientific Computing* 106, Article 18 (2026), 7 pages. doi:10.1007/s10915-025-03130-6
- [6] Przemysław Bogacki and Lawrence F. Shampine. 1989. A 3(2) Pair of Runge–Kutta Formulas. *Applied Mathematics Letters* 2, 4 (1989), 321–325. doi:10.1016/0893-9659(89)90079-7
- [7] John R. Dormand and Peter J. Prince. 1980. A Family of Embedded Runge–Kutta Formulae. *J. Comput. Appl. Math.* 6, 1 (1980), 19–26. doi:10.1016/0771-050X(80)90013-3
- [8] Ernst Hairer, Syvert P. Nørsett, and Gerhard Wanner. 1993. *Solving Ordinary Differential Equations I: Nonstiff Problems* (2 ed.). Springer Series in Computational Mathematics, Vol. 8. Springer, Berlin, Heidelberg. doi:10.1007/978-3-540-78862-1
- [9] Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, et al. 2020. Array Programming with NumPy. *Nature* 585 (2020), 357–362. doi:10.1038/s41586-020-2649-2
- [10] Rasha Al Jahdali, Lisandro Dalcin, and Matteo Parsani. 2022. On the Performance of Relaxation and Adaptive Explicit Runge–Kutta Schemes for High-Order Compressible Flow Simulations. *J. Comput. Phys.* (2022). doi:10.1016/j.jcp.2022.111333
- [11] David I. Ketcheson. 2019. Relaxation Runge–Kutta Methods: Conservation and Stability for Inner-Product Norms. *SIAM J. Numer. Anal.* 57, 6 (2019), 2850–2870. doi:10.1137/19M1263662
- [12] Nicholas Metropolis and Stanislaw Ulam. 1949. The Monte Carlo Method. *J. Amer. Statist. Assoc.* 44, 247 (1949), 335–341. doi:10.1080/01621459.1949.10483310
- [13] Christopher Rackauckas and Qing Nie. 2017. DifferentialEquations.jl – A Performant and Feature-Rich Ecosystem for Solving Differential Equations in Julia. *Journal of Open Research Software* 5, 1 (2017), 15. doi:10.5334/jors.151
- [14] Hendrik Ranocha and David I. Ketcheson. 2020. Relaxation Runge–Kutta Methods for Hamiltonian Problems. *Journal of Scientific Computing* 84, 1, Article 17 (2020). doi:10.1007/s10915-020-01277-y
- [15] Hendrik Ranocha, Lajos Lóczi, and David I. Ketcheson. 2020. General Relaxation Methods for Initial-Value Problems with Application to Multistep Schemes. *Numer. Math.* 146 (2020), 875–906. doi:10.1007/s00211-020-01158-4
- [16] Hendrik Ranocha, Mohammed Sayyari, Lisandro Dalcin, Matteo Parsani, and David I. Ketcheson. 2020. Relaxation Runge–Kutta Methods: Fully Discrete Explicit Entropy-Stable Schemes for the Compressible Euler and Navier–Stokes Equations. *SIAM Journal on Scientific Computing* 42, 2 (2020), A612–A638. doi:10.1137/19M1263480

- [17] Marcin Rogowski, Lisandro Dalcin, Matteo Parsani, and David E. Keyes. 2022. Performance Analysis of Relaxation Runge–Kutta Methods. *The International Journal of High Performance Computing Applications* (2022). doi:10.1177/10943420221085947
- [18] Gustaf Söderlind. 2006. Time-Step Selection Algorithms: Adaptivity, Control, and Signal Processing. *Applied Numerical Mathematics* 56, 3–4 (2006), 488–502. doi:10.1016/j.apnum.2005.04.026
- [19] Gustaf Söderlind and Lina Wang. 2006. Adaptive Time-Stepping and Computational Stability. *J. Comput. Appl. Math.* 185, 2 (2006), 225–243. doi:10.1016/j.cam.2005.03.008
- [20] John Verzani. 2020. Roots.jl: Root Finding Functions for Julia. <https://github.com/JuliaMath/Roots.jl>.
- [21] Pauli Virtanen, Ralf Gommers, Travis E. Oliphant, Matt Haberland, Tyler Reddy, David Cournapeau, et al. 2020. SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python. *Nature Methods* 17 (2020), 261–272. doi:10.1038/s41592-019-0686-2